

Day 12 — Window Functions

ROW_NUMBER, RANK, LAG/LEAD, Running Totals & Moving Averages — Full Detail

This is a high-priority interview topic. All examples use an expanded orders table with several orders per customer, so ranking and comparison functions have something meaningful to work with:

```
orders
-----
order_id | customer_id | order_date | amount
1         | 101         | 2026-01-05 | 500
2         | 101         | 2026-01-12 | 800
3         | 101         | 2026-01-20 | 800
4         | 102         | 2026-01-08 | 300
5         | 102         | 2026-01-15 | 650
6         | 103         | 2026-01-10 | 1200
```

1. What Is a Window Function? (Theory)

A window function performs a calculation across a set of related rows - but unlike GROUP BY (Day 9), it does **not collapse those rows into one**. Every original row stays in the result, with the calculated value attached alongside it.

The critical difference from GROUP BY:

Approach	Behavior
GROUP BY	Collapses many rows into one row per group - detail rows are lost
Window Function	Keeps every row, and attaches a calculated value (rank, total, previous value, etc.) to each one

Example of the difference: "total revenue per customer" (GROUP BY) gives you 3 rows for 3 customers. "Each order alongside that customer's running total" (window function) gives you all 6 order rows, each annotated with its own running total.

Key takeaway: *Reach for GROUP BY when you want a summary. Reach for a window function when you want to keep every detail row but still compare it against its group.*

2. The OVER() Clause

Every window function is followed by OVER(. . .), which defines the "window" of rows the function should look at. It has two optional parts:

- **PARTITION BY** - splits rows into groups (like GROUP BY, but without collapsing rows)
- **ORDER BY** - defines the order rows are processed in *within* each partition (needed for ranking, LAG/LEAD, running totals)

```
SOME_FUNCTION() OVER (  
    PARTITION BY customer_id    -- optional: reset per customer  
    ORDER BY order_date        -- optional: defines row order  
)
```

Key takeaway: *Think of PARTITION BY as "restart the calculation for each group" and ORDER BY (inside OVER) as "the sequence to process rows in" - both are optional, and omitting PARTITION BY means the whole table is treated as one big group.*

3. ROW_NUMBER()

Assigns a unique, sequential number to each row within its partition, starting at 1. No two rows ever get the same number, even if their values are tied.

```
SELECT  
    customer_id,  
    order_date,  
    amount,  
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS row_num  
FROM orders;
```

Result:

customer_id	order_date	amount	row_num
101	2026-01-05	500	1
101	2026-01-12	800	2
101	2026-01-20	800	3
102	2026-01-08	300	1
102	2026-01-15	650	2
103	2026-01-10	1200	1

Notice numbering **restarts at 1 for each customer** because of PARTITION BY customer_id.

Common real use: *combined with a filter, this finds "the Nth row per group" - e.g., each customer's very first order (WHERE row_num = 1).*

4. RANK() vs. DENSE_RANK()

Both assign a rank based on ORDER BY, and - unlike ROW_NUMBER - both give **tied rows the same rank**. They only differ in what happens *after* a tie.

Example - ranking customer 101's orders by amount:

```

SELECT
    order_date,
    amount,
    ROW_NUMBER() OVER (ORDER BY amount DESC) AS row_num,
    RANK() OVER (ORDER BY amount DESC) AS rank_val,
    DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rank_val
FROM orders
WHERE customer_id = 101;

```

Result (note the tie between the two 800s):

order_date	amount	row_num	rank_val	dense_rank_val
2026-01-12	800	1	1	1
2026-01-20	800	2	1	1
2026-01-05	500	3	3	2

The difference, spelled out:

- **ROW_NUMBER:** 1, 2, 3 - always unique, ties broken arbitrarily
- **RANK:** 1, 1, 3 - ties share a rank, but the next rank *skips* ahead (no rank 2 exists)
- **DENSE_RANK:** 1, 1, 2 - ties share a rank, and the next rank does *not* skip

Key takeaway: Choose based on what "next place" should mean. Sports leaderboards usually use **RANK** (two 1st-place ties mean the next person is 3rd). **DENSE_RANK** is more common when you want a clean, gap-free set of tiers (e.g., "top 3 price tiers").

5. LAG() and LEAD()

LAG () looks back at a *previous* row's value; LEAD () looks ahead at a *following* row's value - both within the same partition and order, without needing a self-join (Day 10).

```

SELECT
    customer_id,
    order_date,
    amount,
    LAG(amount) OVER (
        PARTITION BY customer_id ORDER BY order_date
    ) AS previous_amount,
    LEAD(amount) OVER (
        PARTITION BY customer_id ORDER BY order_date
    ) AS next_amount
FROM orders;

```

Result (for customer 101):

order_date	amount	previous_amount	next_amount
2026-01-05	500	NULL	800
2026-01-12	800	500	800
2026-01-20	800	800	NULL

Notice the first row has NULL for previous_amount (nothing comes before it), and the last row has NULL for next_amount (nothing comes after it).

Very common real use - calculating change between orders:

```
SELECT
    customer_id,
    order_date,
    amount,
    amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS change_from_last_order
FROM orders;
```

Key takeaway: LAG/LEAD replace what used to require a messy self-join. Any "compare this row to the previous/next row" question - month-over-month change, days-since-last-order - is a LAG/LEAD question.

6. Running Totals

A running total (cumulative sum) adds up values progressively as you move through the ordered rows. Built using SUM() as a window function with ORDER BY.

```
SELECT
    customer_id,
    order_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS running_total
FROM orders;
```

Result (for customer 101):

order_date	amount	running_total
2026-01-05	500	500
2026-01-12	800	1300 -- 500 + 800
2026-01-20	800	2100 -- 500 + 800 + 800

Why this works: When ORDER BY is present inside OVER() with an aggregate function like SUM, it defaults to summing "all rows from the start of the partition up to and including the current row" - not the whole partition at once.

Key takeaway: This single pattern - aggregate function + PARTITION BY + ORDER BY - is exactly how running totals, cumulative counts, and year-to-date sums are built in real dashboards.

7. Moving Averages

A moving average smooths out data by averaging only a defined window of nearby rows (e.g., "the last 3 orders") instead of everything up to that point. This needs an explicit frame using ROWS BETWEEN.

```

SELECT
    customer_id,
    order_date,
    amount,
    AVG(amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) AS moving_avg_2_orders
FROM orders;

```

Breaking down the frame clause:

- 1 PRECEDING - include the 1 row before the current row
- CURRENT ROW - include the current row itself
- Together: a 2-row moving window (current + 1 before) - adjust the number for a wider average, e.g. 2 PRECEDING for a 3-row average

Result (for customer 101):

```

order_date | amount | moving_avg_2_orders
2026-01-05 | 500    | 500.0    -- only 1 row available, avg = itself
2026-01-12 | 800    | 650.0    -- avg of 500 and 800
2026-01-20 | 800    | 800.0    -- avg of 800 and 800

```

Key takeaway: A running total (Section 6) uses an expanding window (grows with every row); a moving average uses a fixed-size, sliding window. ROWS BETWEEN is what controls exactly which nearby rows are included.

Day 12 Summary — Window Functions

Function	What It Does
ROW_NUMBER()	Unique sequential number per row within a partition
RANK()	Rank with ties sharing a value; next rank skips ahead
DENSE_RANK()	Rank with ties sharing a value; next rank does not skip
LAG()	Looks back at a previous row's value
LEAD()	Looks ahead at a following row's value
SUM() OVER (...ORDER BY)	Running/cumulative total
AVG() OVER (...ROWS BETWEEN)	Moving average over a fixed sliding window

Overall takeaway: Window functions solve the exact problems that used to require messy self-joins or correlated subqueries (Day 10-11) - ranking, comparing to neighboring rows, and cumulative calculations - while keeping every original row intact. PARTITION BY, ORDER BY, and an optional frame (ROWS BETWEEN) are the three levers that control every window function you'll write.